A Simple and Effective Evolutionary Algorithm for the Vehicle Routing Problem

*Christian Prins**

* University of Technology of Troyes, Laboratory for Optimization of Industrial Systems
12 Rue Marie Curie, BP2060, F-10010 Troyes Cedex (France)
Email: prins@univ-troyes.fr

1 Introduction

The NP-hard Vehicle Routing Problem (VRP) is defined on an undirected network $G = (V, E)$ with a node set $V = \{0, 1, \ldots, n\}$ and an edge set E. Node 0 is a depot with identical vehicles of capacity Q. Each client node $i > 0$ has a non-negative demand q_i and each edge $[i, j]$ has a non-negative cost $c_{ij} = c_{ji}$. The VRP consists of determining a set of vehicle trips of minimum total cost, such that each trip starts and ends at the depot, each client is visited exactly once, the total demand handled by any vehicle does not exceed Q, and the cost of any trip does not exceed a given upper limit L. In this paper, the number of vehicles is a decision variable and the costs are real numbers.

The VRP is very hard in practice, since some Euclidean instances with 75 nodes are not yet solved to optimality [8]). Heuristics are then required for solving practical instances. Several constructive heuristics are reviewed in [2][5], they include classics from Clarke and Wright, Mole and Jameson, and Gillett and Miller. The best metaheuristics for the VRP are powerful tabu search (TS) algorithms published in the 90's and surveyed in [3][4]. They easily outperform other metaheuristics like simulated annealing (SA), genetic algorithms (GA) and ant algorithms. Contrary to the VRPTW, the VRP is not well solved by GAs. Van Breedam [9] reports comparable results with a GA, a TS and a SA, but evaluated on his own set of instances and not compared with published metaheuristics or standard sets of benchmarks. Schmitt [7] designed a GA which proved better than the Clarke and Wright heuristic on the Christofides instances [2], but worse than some constructive heuristics combined with improvement procedures.

This paper describes a relatively simple, yet effective, hybrid GA which outperforms all TS algorithms published except one. Section 2 to 6 present the basic components: chromosomes, evaluation, population, reproduction step, mutation by local search, replacement method and stopping criteria. A preliminary computational evaluation is given in section 7.

2 Chromosomes and evaluation

We code any VRP solution as a sequence (permutation) S of n clients *without trip delimiters*. It can be interpreted as the order in which a vehicle must visit all clients, assuming the same vehicle performs all trips in turn. This simple encoding is appealing because *there obviously exists one optimal sequence*. We let the intrinsic parallelism of the GA find this sequence and we use a splitting procedure *to split optimally any sequence into trips and recover the corresponding VRP solution*, using an auxiliary directed and weighted graph $H = (X, A, W)$.

We first construct X with $n+1$ nodes indexed from 0 to n. Then we add one arc (i,j), $i<j$, if subsequence S_{i+1} to S_j (included) of clients corresponds to a feasible trip: total demand not greater than Q, total cost of the trip $(0, S_{i+1}, S_{i+2}, \ldots, S_j, 0)$ not greater than L. The arc has a weight w_{ij} equal to the trip cost. For instance, we create arc $(0,2)$ if a trip reduced to clients 1 and 2 is feasible. Since H is circuitless, a min-cost path μ from node 0 to node n can be computed using Bellman's algorithm. As each arc of μ corresponds to one trip, we can extract the underlying VRP solution, whose total cost is equal to the cost of μ. Our GA uses the splitting procedure to evaluate each chromosome. The fitness is simply the total cost of the resulting VRP solution. The evaluation is reasonably fast because H and μ can be built in $O(n^2)$ and because trip details are not required (except to output solutions when the GA stops): we need the cost of μ, not the actual composition of the path.

3 Population structure and initialization

The population is an array P of nc chromosomes, sorted in decreasing cost order. At the beginning, we run three constructive heuristics: those from Clarke and Wright (parallel version + 3-opt), Mole and Jameson, and Gillett and Miller. We use our own implementations for these heuristics described for instance in [2]. Each solution is converted into a chromosome by concatenating its trips. The splitting procedure is applied to these chromosomes because its improves sometimes the total cost found by the heuristics. The population is completed by $nc-3$ random sequences of clients, immediately evaluated by the splitting procedure.

Clones (identical solutions) are forbidden in P to have a better dispersion of solutions and to diminish the risk of premature convergence. Clone detection being time-consuming, we adopt a more restrictive but faster policy in which the costs of any two solutions must be spaced at least by a constant $\Delta > 0$ (we call this the Δ-*property*). In a paper on the open-shop scheduling problem [6], we successfully tried this idea with $\Delta = 1$ (solutions with distinct costs). When building each initial random chromosome $P(k)$, we try up to mnt times (50 for instance) to get one satisfying the property. In case of failure, P is truncated to $k-1$ chromosomes, but this occurs only when nc is too large.

4 Reproduction step and crossover

Parents are chosen by *binary tournament selection*. We first randomly select two solutions from the population. We then select from these two the least cost solution to be the first parent $P1$. This procedure is repeated to get the second parent $P2$. Since our chromosomes do not contain trip delimiters, we can apply classical crossovers like the *order crossover* (OX) or *linear order crossover* (LOX), and evaluate children using the splitting procedure. The reproduction step ends by randomly keeping only one child C and by discarding the other. This policy works slightly better than keeping two children or the best one. The child is evaluated with the splitting procedure.

A given VRP solution can be encoded as different chromosomes, depending in which order its trips are concatenated. So there is no reason to distinguish a first or last client in the sequence, as LOX does. We selected OX for the GA, after some testing confirming its superiority over LOX. For two parents $P1$ and $P2$, OX draws two random subscripts p and q with $1 \le p \le q \le n$. To build child $C1$, it copies the string $P1(p)-P1(q)$ into $C1(p)-C1(q)$. Finally, it scans $P2$ in a circular way from $q+1 \pmod n$ and copies each element not yet taken to fill $C1$ circularly, starting from $q+1 \pmod n$. The roles of $P1$ and $P2$ are exchanged to get the other child $C2$.

5 Local search as mutation operator

We mutate with a fixed rate p_m the child C kept after OX. The mutation operator is in fact a *local search LS*, the GA is then *hybrid*. Before applying LS, the child is converted into a VRP solution by analyzing the shortest path μ computed by the splitting procedure. Compared to some TS algorithms, LS needs not be sophisticated because a relative weakness can be compensated by the intrinsic parallelism of the GA. Each iteration of LS scans in $O(n^2)$ all possible pairs of distinct nodes (u, v). Nodes u and v can be clients or the depot and belong to the same trip or to distinct trips. For each pair (u, v), the following moves are tested in turn (x is the successor of u and y the successor of v along their respective trips):

– Move 1: if u is a client node, move u after v,
– Move 2: if u and x are client nodes, move (u, x) after v,
– Move 3: if u and x are client nodes, move (x, u) after v,
– Move 4: if u and v are client nodes, permute u and v,
– Move 5: if u, x and v are clients nodes, permute (u, x) with v,
– Move 6: if (u, x) and (v, y) are client nodes, permute (u, x) and (v, y),
– Move 7: if (u, x) and (v, y) are non adjacent in the same trip, replace them by (u, v) and (x, y)
– Move 8: if (u, x) and (v, y) are in distinct trips, replace them by (u, v) and (x, y),
– Move 9: if (u, x) and (v, y) are in distinct trips, replace them by (u, y) and (x, v).

Move 7 corresponds to 2-opt, while moves 8–9 generalize 2-opt to different trips. Each iteration stops at the first improving move. The process is repeated until no further saving can be found. Some trips may become empty, they are removed at the end of LS. The solution of LS is converted into a chromosome MC by concatenating the lists of clients of its trips. Like for the heuristic solutions put into the initial population, MC is re-evaluated with the splitting procedure because this may improve LS slightly by splitting differently the same sequence.

6 Replacement method and stopping criteria

The replacement is *incremental*. We draw a chromosome $P(k)$ below the median, *i.e.* $k \leq \lfloor nc/2 \rfloor$. After a mutation by LS, $P(k)$ is replaced by MC if this respects the Δ-property. If not, or when LS is not applied, we replace $P(k)$ by the child C obtained by OX, provided the Δ-property holds. If this fails too, the current GA iteration is said *unproductive* and is discarded. P is re-sorted after each replacement. This technique preserves the best solution $P(nc)$ and allows a good child to reproduce immediately. The GA stops after a maximum number of iterations mni, after a maximal number of unproductive iterations $mnui$, or when it reaches the optimum cost OPT known for some instances (in that case $P(nc)$ is of course optimal).

7 Computational evaluation

The GA is implemented in Delphi on a 500 MHz PC under Windows 95 and already tested on the 14 instances of Christofides *et al.* [2] (see *http://mscmga.ms.ic.ac.uk/jeb/orlib* for instance). These instances are euclidean VRPs with 50 to 199 clients. It is permitted to travel between any two nodes i and j, the cost c_{ij} being the euclidean distance. 7 instances include a maximum route length L. The solutions found for these instances by 12 TS algorithms are listed in [3][4]. All best-known solutions are due to these algorithms.

The GA performs best with small populations of $nc = 30$ distinct solutions, $\Delta = 2$ and a mutation by local search with rate $p_m = 0.05$. nc is small to avoid losing too much time in unproductive crossovers (only 5 to 15% for $nc = 30$). The GA performs a main phase ending after $mni = 30000$ productive crossovers, $mnui = 10000$ crossovers without improving the best solution, or when OPT is reached (it

is known only for problems 1 and 12). If OPT is not reached, the GA is restarted for 10 short phases with $mni = mnui = 2000$ and p_m pushed up to 0.1. At each restart, the best solution is kept but 25% of the population is replaced by random chromosomes, using a special *partial replacement procedure* proposed by Cheung, Langevin and Villeneuve [1] in a GA for a facility location problem.

The three first columns of Table 1 show the problem numbers, the numbers of client nodes and the best-known solution values from [3][4]. Col. 4 gives the GA results with the standard parameters, and col. 5 the computing times in seconds. Col. 6 gives the best result we have found so far with various settings. When the GA starts, the best solution in population is on average at 6.8% from the best-known solution. Using one setting, it stops with an average deviation of only 0.23% and retrieves 8 best-known solutions (boldface). Using several settings, it reaches 10 best-known solutions with a deviation of 0.08%. Our implementation is not yet optimized and running times can be improved. For instance, solutions in col. 4 are found during restarts only for problems 4, 5, 10, 13. For all other problems, $P(nc)$ is achieved in the main phase and CPU time could be multiplied by 0.6 without restarts (max. $30,000$ crossovers instead of $50,000$). Moreover, the GA is always within 1% from best-known solution after 1000 crossovers (roughly 1/50 of the given running time).

Problem	Clients	Best-known	GA (std. setting)	CPU (s)	GA (best setting)
1	50	*524.61	**524.61**	< 1	**524.61**
2	75	835.26	**835.26**	184	**835.26**
3	100	826.14	**826.14**	497	**826.14**
4	150	1028.42	1031.63	1955	1030.46
5	199	1291.45	1300.23	3364	1296.39
6	50	555.43	**555.43**	106	**555.43**
7	75	909.68	912.30	315	**909.68**
8	100	865.94	**865.94**	664	**865.94**
9	150	1162.55	1164.25	2319	**1162.55**
10	199	1395.85	1420.20	5279	1402.75
11	120	1042.11	**1042.11**	783	**1042.11**
12	100	*819.56	**819.56**	7	**819.56**
13	120	1541.14	1542.97	2143	1542.86
14	100	866.37	**866.37**	591	**866.37**

Table 1: GA results for Christofides instances

The comparison with TS algorithms surveyed in [3][4] is difficult, because the authors seldom give their best-known solutions for one standard setting of parameters, as recommended nowadays. Generally, they give best solutions obtained with different settings and do not mention corresponding CPU times. Even when given, running times are anyway difficult to compare because various computers are used, even parallel ones ! Nevertheless, we try in Table 2 to compare on Christofides instances our GA, 10 TS and the best SA algorithm published for the VRP. Due to lack of space, we cannot give the references of these algorithms, but the reader will find all of them in [3][4]. The criteria are the average distance in % to best-known solutions, the number of best-known solutions retrieved, and the average computation time in minutes. The table (sorted in increasing order of distance to best solutions) shows very encouraging results: only one published TS out of 10 does better than the GA. Even with one single parameter setting, the GA would be at rank 4, with an average deviation of 0.23%.

8 Conclusion

This paper presents the first hybrid GA for the VRP competing with recent TS algorithms, according to a preliminary testing on Christofides instances. Despite simple neighborhoods, the GA is effective thanks some simple ideas. A possible premature convergence due to local search is prevented by using

Kind	Authors	Year	Average percent above best-known	Number of best-known	Average CPU time (min)
TS	Taillard	1993	0.05	12	unknown
GA	Prins (this paper)	2001	0.08	10	21.7
TS	Xu-Kelly (7 problems)	1996	0.10	5	103.0
TS	Gendreau *et al.*	1994	0.20	8	unknown
TS	Taillard	1992	0.39	6	unknown
TS	Rego & Roucairol	1996	0.55	6	unknown
TS	Gendreau *et al.*	1991	0.68	5	unknown
TS	Rego & Roucairol	1996	0.77	4	unknown
TS	Gendreau *et al.*	1994	0.86	5	46.8
TS	Osman	1993	1.01	4	26.1
TS	Osman	1993	1.03	3	34.0
SA	Osman	1993	2.09	2	unknown

Table 2: Overall comparison of VRP metaheuristics for the 14 Christofides instances

small populations of distinct solutions (Δ-property). Three classical heuristics provide good starting points. The decrease of the objective function is accelerated by the incremental management of P and the partial replacement technique in the restart phases. Thanks to the splitting procedure, we can forget trip limits and use simple chromosomes and crossovers, like for the TSP. Our objectives are now to evaluate this GA on more data sets like the large scale instances of Xu and Kelly discussed in [4] and to diminish its computation time. When sending this abstract, we just improved the best known solution of Kelly's 13^{th} problem, by finding 878.46 instead of 881.04.

References

[1] B.K.S. Cheung, A. Langevin, B. Villeneuve. High performing evolutionary techniques for solving complex location problems in industrial system design. To appear in *JIM*.

[2] N. Christofides, A. Mingozzi, P. Toth. The vehicle routing problem. Chapter 11 in *Combinatorial Optimization*, edited by the same authors and C. Sandi, Wiley, 1979.

[3] M. Gendreau, G. Laporte, J-Y. Potvin. Metaheuristics for the vehicle routing problem. *GERAD research report G-98-52*, Montréal, Canada, 1998.

[4] B.L. Golden, E.A. Wasil, J.P. Kelly, I.M. Chao. The impact of metaheuristics on solving the vehicle routing problem: algorithms, problem sets, and computational results. Chapter 2 in *Fleet management and logistics*, T.G. Crainic and G. Laporte (ed.), Kluwer, 1998.

[5] G. Laporte, M. Gendreau, J.Y. Potvin, F. Semet. Classical and modern heuristics for the vehicle routing problem, *Int. Trans. in Oper. Res.*, 7, 285–300, 2000.

[6] C. Prins. Competitive genetic algorithms for the open-shop scheduling problem, *Math. Meth. Oper. Res. (MMOR)*, 52(3), 389–411, 2000.

[7] L.J. Schmitt. An evaluation of a genetic algorithmic approach to the VRP. *Working paper*, Dep. of Information Technology Management, Christian Brothers University, Memphis, USA, 1995.

[8] P. Toth, D. Vigo. Exact solution of the vehicle routing problem. Chapter 1 in *Fleet management and logistics*, edited by T.G. Crainic and G. Laporte, Kluwer, 1998.

[9] A. Van Breedam. An analysis of the effect of local improvement operators in GA and SA for the vehicle routing problem. *RUCA Working Paper 96/14*, University of Antwerp, Belgium, 1996.